

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

ABSTRACT

- **문제 제기:** Transformer는 NLP에서 표준이 되었지만, CV(컴퓨터 비전)에서는 CNN 의존도가 여전히 높음.

→ **Vision** 분야에서는 아직 Transformer 구조가 활발하게 적용되는 단계는 아니었는데, 기껏해야 **convolution networks**와 같이 쓰이거나, 일부 구성요소를 대체하는 정도.

여전히 ResNet과 같은 CNN 기반 백본 모델들이 이미지 인식 분야에서 Sota를 보여주고 있었다.

- **주장:** CNN 없이도, 이미지를 패치 단위로 나눠 **순수 Transformer**에 넣으면 이미지 분류가 잘 된다.
→ 이 때 **image patches**를 **sequence**로 바라봄.
- **조건:** 대규모 데이터셋으로 사전학습(pre-training) → 작은 데이터셋에 전이학습(transfer)하면 성능 ↑
- **결과:** 최신 CNN(State-of-the-Art)과 비슷하거나 더 좋은 성능을 달성하면서, 계산 자원도 더 적게 든다.

→ 요약하면: “**Vision**에도 Transformer만 써도 된다. CNN 필요 없다. (단, 큰 데이터셋에서 학습해야 한다)”

Self-attention기반 아키텍처는 NLP분야의 태스크에서 보편적으로 쓰이는 모델(대표적으로 트랜스포머).

일반적으로 Large text corpus에 사전학습시킨 다음, smaller task-specific dataset에 전이학습을 시키는 접근법이 널리 사용된다.

Transformer 기반 모델들은 아래와 같은 장점을 가지기에 더욱 널리 사용되고 있으며, 모델과 데이터셋이 증가함에 따라 성능 또한 포화상태에 이를 기미가 아직은 보이지 않는다.

Transformer-based 모델의 장점

1. 연산이 효율적이다(computational efficiency).

병렬 연산: Transformer의 핵심인 셀프 어텐션(self-attention) 메커니즘은 문장의 각 단어를 한 번에(동시에) 처리한다. 이와 달리 RNN은 단어들을 순서대로(순차적으로) 하나씩 처리해야 했다. 따라서 GPU 같은 현대 하드웨어는 병렬 연산에 최적화되어 있으므로, Transformer는 RNN에 비해 훨씬 빠르게 훈련되고 추론을 할 수 있다.

2. 확장성이 좋다(scalability). 특히 input sequence의 길이에 구애받지 않는다.

입력 길이 무관: Transformer는 입력 시퀀스의 길이가 길어져도 처리에 제약이 없다. RNN은 시퀀스가 길어질수록 앞선 정보를 잊어버리는 '장기 의존성(long-term dependency)' 문제가 있었지만, Transformer는 셀프 어텐션을 통해 모든 단어 간의 관계를 동시에 계산하므로 입력 길이와 상관없이 전체 문맥을 파악할 수 있다.

모델 크기 확장: 이러한 효율성과 유연성 덕분에 Transformer는 모델의 파라미터(매개 변수) 수를 크게 늘려도 안정적으로 훈련할 수 있다. 이것이 1,000억 개 이상의 파라미터를 가진 거대 모델이 등장하게 된 이유이며, 데이터셋이 커질수록 성능이 계속 향상되는 '규모의 법칙'이 적용된다.

본 논문은 이러한 한계를 극복하고자 이미지를 패치로 분할하고 이들을 선형 시퀀스로 변환하여 Transformer 구조에 입력하는 방법을 제안했다. 이 방법은 이미지 패치들을 NLP에서의 토큰처럼 다루며, 이를 통해 이미지 분류 작업에 적용한다.

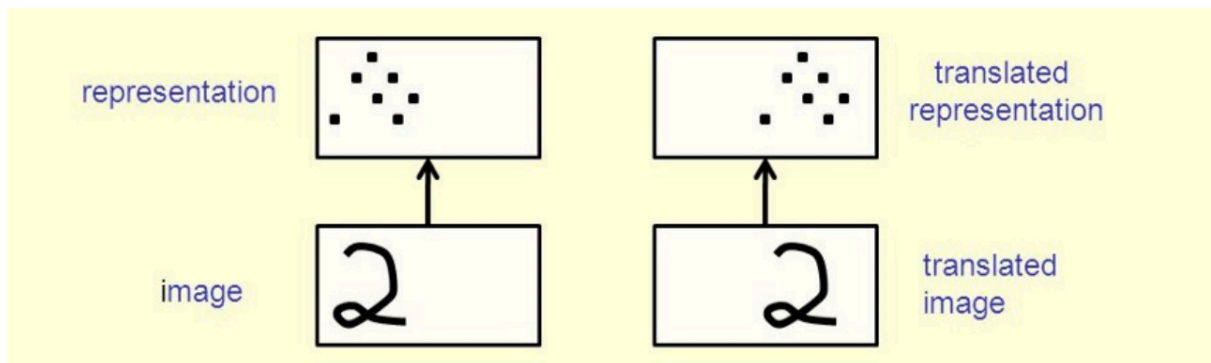
→ 즉, ViT에서 하나의 이미지 패치는 하나의 **token**으로 작동한다.

그러나 ResNet과 같은 크기의 ImageNet 데이터 세트를 사용하여 학습했을 때, Transformer 기반 모델은 약간 떨어지는 성능을 보였다. 이는 CNN이 가지는 Translation equivariance와 locality 같은 본질적인 특성, 즉 inductive biases가 부족하기 때문으로 분석된다.

Inductive biases(귀납적 편향)

Inductive biases는 딥러닝 알고리즘의 일반화를 돕기 위해 추가적으로 가정하는 개념.

모든 딥러닝 알고리즘에는 이러한 가정이 내재되어 있으며, 특히 **CNN에는 Translation equivariance와 locality와 같은 중요한 가정들이 존재한다**. CNN은 $k \times k$ 크기의 필터를 사용하여 이미지의 국소적인 부분에 연산을 수행한다. 이러한 연산 방식은 Locality의 개념을 반영하며, 이를 통해 **이미지가 이동해도 같은 특징을 인식할 수 있는 Translation equivariance** 특성을 가지게 된다.



1. Translation Equivariance (이동 등가성)

- **뜻:** 입력 이미지가 단순히 옆으로 움직이면, 출력 표현(representation)도 똑같이 옆으로 움직인다.
- 예시 (그림 왼쪽 → 오른쪽):
 - 왼쪽 "2" 이미지를 CNN에 넣으면 특정 feature map(검은 점들) 나옴.
 - 오른쪽 "2" 이미지를 **살짝 옮긴 버전**을 넣으면 → feature map도 똑같이 **살짝 옮겨진 버전**으로 나옴.
- 이유: CNN의 필터가 **이미지 전체에 공유(weight sharing)** 되기 때문.
 - 같은 모양이 어느 위치에 있어도 같은 방식으로 탐지됨.

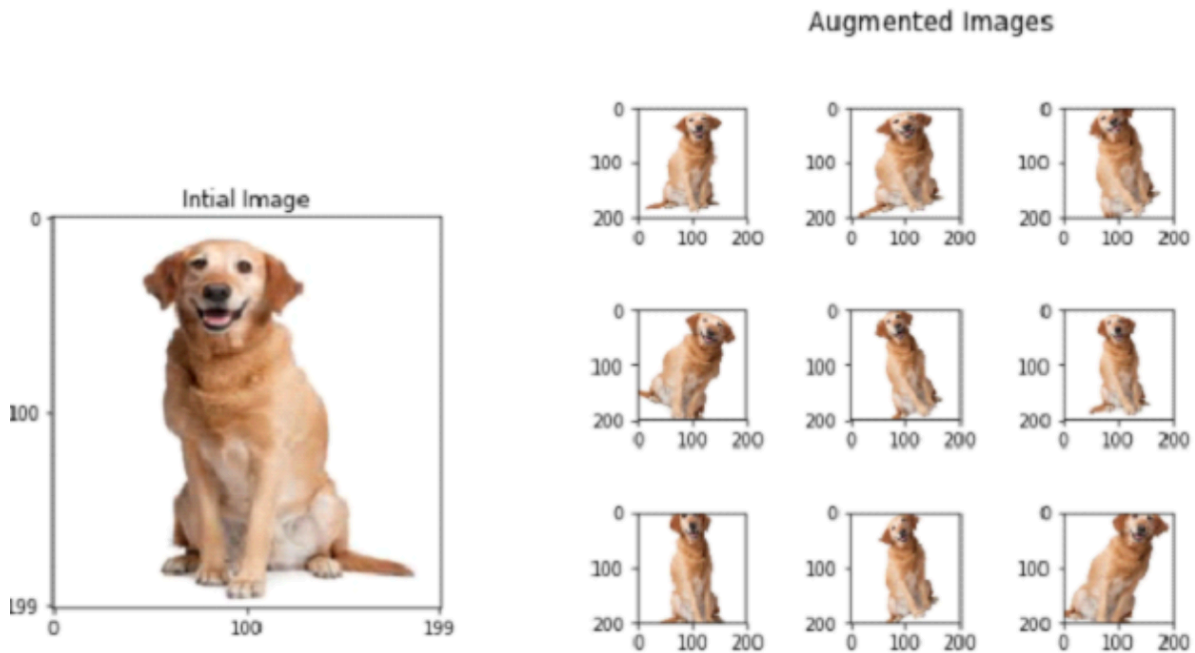
즉 "입력을 이동시키면, 출력도 똑같이 이동한다."

2. Translation Invariance (이동 불변성)

- **뜻:** 입력이 조금 움직여도 최종적으로는 **같은 클래스**로 분류된다.
- 과정:
 1. CNN의 Convolution은 equivariance(등가성)를 보장.
 2. 하지만 representation 위치가 바뀌면 결과가 달라질 수 있음.
 3. 그래서 **Pooling + Softmax** 같은 과정을 넣어 위치 변화를 줄임.

- Max Pooling은 주변에서 가장 강한 특징만 가져오기 때문에, 입력이 약간 이동해도 결과는 안정적.
- 결과: 입력 "2"가 왼쪽/오른쪽으로 조금 움직여도 최종 분류는 똑같이 "2"로 나옴.
즉 "입력이 움직여도, 최종 결과(클래스)는 변하지 않는다."

3. 한계



- CNN이 translation(이동)에는 강하지만,
- rotation(회전), scale(크기 변화)에는 그 성질이 깨진다.
 - 예: "2"를 90도 회전시키면 CNN이 잘 인식 못할 수도 있음.
 - 이를 해결하려면 Data Augmentation, Spatial Transformer, 혹은 최근엔 Vision Transformer처럼 전역 구조를 학습하는 방법이 필요.

그럼 반대로 왜 transformer가 대규모 데이터 셋에서는 더 높은 성능이 나왔을까?

1. 작은 데이터에서는 CNN이 유리한 이유

- **CNN**은 "내장 규칙(필터)"이 있어서 이미지를 볼 때 근처 픽셀끼리 중요한 관계가 있다고 가정하고 시작함.
- 예를 들어, 얼굴 사진을 본다고 하면 CNN은 눈-코-입은 가까이 붙어 있다는 걸 구조적으로 이미 알고 있는 셈임. 그래서 데이터가 조금만 있어도 금방 "얼굴"을 인식할 수 있

다.

- 반대로 **Transformer**는 이런 규칙이 하나도 없기 때문에 **모든 픽셀을 다 똑같이 보고 스스로 규칙을 찾아야 함.**

→ 데이터가 적으면 규칙을 제대로 못 배우고 헛갈려서 성능이 떨어짐.

2. 큰 데이터에서는 Transformer가 더 유리한 이유

- CNN은 계속 "근처에서만 특징 찾기 → 그걸 쌓아서 전체 이해"라는 **고정된 방식**만 사용할 수 있다.
 - 데이터가 아무리 많아도 이 구조적인 한계 때문에 성능 향상이 점점 줄어듦. (포화)
→ CNN은 구조상 **한계치가 정해져 있어서** 성능 곡선이 어느 지점에서 **점점 평평해짐.**
 - Transformer는 처음에는 아무 규칙이 없어서 힘들지만, **데이터가 충분하면 스스로 훨씬 더 다양한 규칙을 학습**할 수 있다.
 - 예를 들어, 멀리 떨어진 두 패치(하늘 끝과 땅 끝)의 관계까지 한 번에 고려 가능.
 - 즉, 데이터가 크면 클수록 Transformer는 제한 없이 계속 성능이 올라감.
-

RELATED WORK

BERT

구조: Transformer **Encoder**만 사용.

학습 방식 (사전학습):

- **Masked Language Modeling (MLM)**

문장에서 단어를 일부 [MASK]로 가리고, 원래 단어를 맞히도록 학습.

예: "나는 [MASK]에 간다" → 정답: 학교

- **Next Sentence Prediction (NSP)**

두 문장이 이어지는지 아닌지를 예측.

특징:

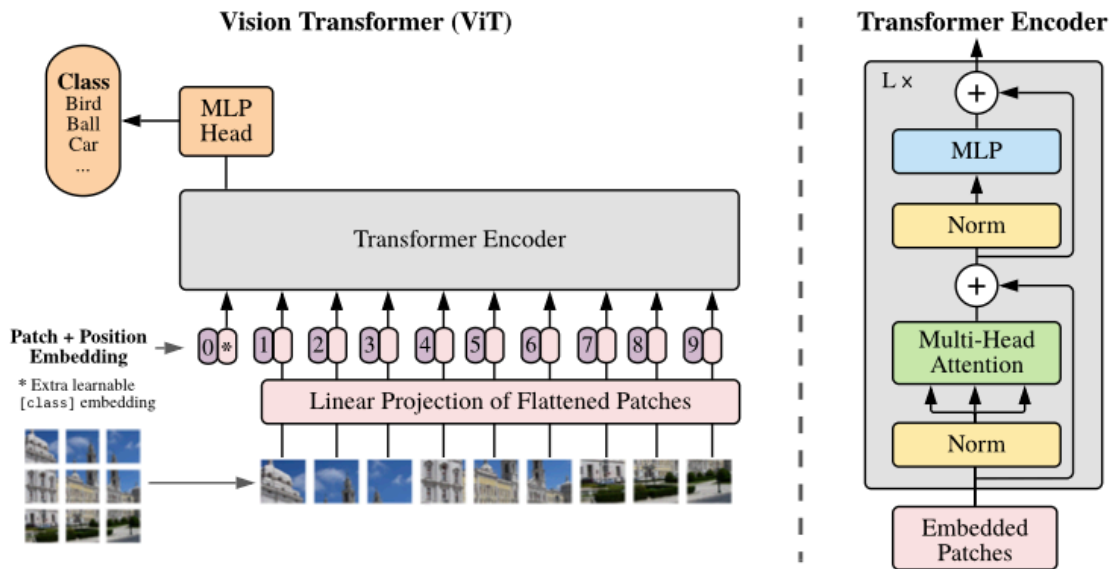
- 문맥을 ****양방향(bidirectional)****으로 모두 고려.
- 작은 데이터셋에도 파인튜닝으로 잘 적용됨.
- 대표적 응용: 문장 분류, QA, NER 등.

BART

- **구조: Transformer Encoder + Decoder (전체 구조)**
 - Encoder는 BERT처럼 양방향, Decoder는 GPT처럼 한쪽 방향(좌→우).
- **학습 방식 (사전학습): Denoising Autoencoder**
 - 입력 문장을 일부 망가뜨린 후(단어 삭제, 순서 섞기 등), 원래 문장을 복원하도록 학습.
 - 즉, "손상된 입력 → 원래 텍스트"를 재구성.
- **특징:**
 - **텍스트 생성**(summarization, translation, paraphrasing)에 강함.
 - BERT의 이해 능력 + GPT의 생성 능력을 합친 하이브리드 구조.

GPT

- **구조: Transformer Decoder만 사용.**
- **학습 방식 (사전학습):**
 - **Language Modeling (LM)**
문장을 왼쪽에서 오른쪽으로 한 단어씩 예측.
예: "나는 학교에" → 다음 단어 예측: 간다
- **특징:**
 - **단방향(unidirectional)** 문맥 → 미래 단어는 못 보고 현재 단어를 예측.
 - 대규모 데이터와 파라미터 확장으로 강력한 성능 발휘 (GPT-2, GPT-3, GPT-4, GPT-5...).
 - 대표적 응용: 대화형 AI, 글쓰기, 코드 생성 등.



Self-Attention이 하는 일

- Transformer의 핵심 = **Self-Attention**
- 원리: 입력에 들어온 모든 요소들이 **서로 얼마나 관련 있는지** 점수를 계산.
 - 예: 문장 "I love dogs"
 - "I" ↔ "love"
 - "I" ↔ "dogs"
 - "love" ↔ "dogs"
 - 모든 단어 쌍의 관계를 계산 → 즉, **쌍(pair)마다 연산**.

Self-Attention = 모든 입력 × 모든 입력 관계 계산.

이미지를 픽셀 단위로 넣는다면?

- 보통 이미지는 크기가 $224 \times 224 = 50,176$ 픽셀.
- Transformer에 픽셀 하나하나를 토큰처럼 넣으면 → 입력 길이 $N = 50,176$.
- Self-Attention 계산량 = N^2 .
- $N = 50,176 \rightarrow N^2 \approx 2.5 \text{ billion (25억)}$
- 즉, 한 장 이미지를 Self-Attention에 넣을 때 → 25억 번의 연산이 필요.
- 게다가 딥러닝에서는 이런 걸 수천~수만 장 반복해야 함.
- 현실적으로 GPU 메모리/시간이 감당 불가.

해결 방안

(1) Local Attention

- 한 픽셀이 주변 픽셀(이웃)이랑만 Attention 계산.
- 마치 CNN의 작은 커널(3×3, 5×5)처럼 근처만 보는 방식.
- 장점: 연산 줄어듦, CNN 역할 비슷하게 가능.

(2) Sparse Attention

- 모든 픽셀 쌍을 다 연결하지 말고, **간격을 두고 일부만 연결**.
- 예시:
 - 이미지를 **큰 블록**으로 나눠서 블록끼리 Attention.
 - 또는 x축·y축 방향(행, 열)으로만 Attention.
- 장점: 전역 정보도 어느 정도 커버하면서 연산 줄임.

한계

- 성능은 꽤 괜찮지만, GPU/TPU 같은 **병렬 하드웨어에 최적화하기 어려움**.
- 이유: GPU는 “행렬 곱” 같은 규칙적이고 반복적인 연산에 특화되어 있음.
- 하지만 Local/Sparse Attention은 “여긴 보고, 여긴 안 보고” 식의 **불규칙한 연산**이라 구현이 복잡하고 속도가 잘 안 나옴.

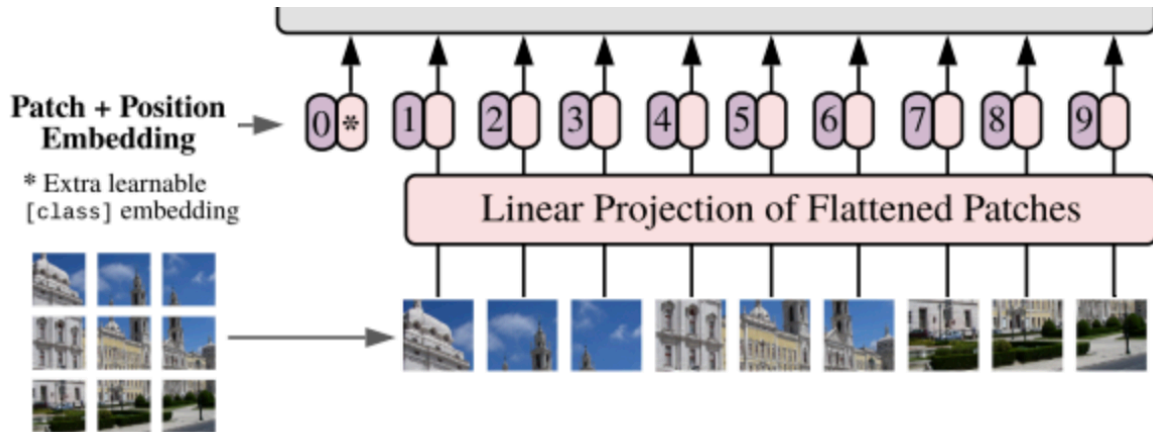
그래서 생각해낸 방안은?

ViT의 아이디어

- 굳이 픽셀 단위로 할 필요 없이 이미지를 그냥 **16×16 패치**로 잘라서, 각 패치를 토큰(단어)처럼 보자.
- 이렇게 하면 입력 개수가 **픽셀 수 → 패치 수**로 확 줄어듦.
- 예: 224×224 이미지를 16×16으로 자르면 → 픽셀 50,000개 → 패치 196개.
- Self-Attention 계산량: **25억 → 38,000**으로 대폭 감소
- 게다가 **표준 Transformer 코드** 그대로 쓰면 되니 GPU/TPU에서 쉽게 학습 가능.

METHOD

3.1. Vision Transformer(ViT)



기본적인 구조는 standard Transformer와 동일하였고 Encoder/Decoder 블록으로 구성.

Transformer의 기본 입력 형식:

NLP에서 Transformer는 **단어 토큰**을 입력으로 받음.

각 단어는 보통 **D차원 벡터**(예: 512차원, 768차원 등)로 표현됨.

즉, Transformer는 **D차원 벡터 시퀀스**만 처리할 수 있음.

따라서 ViT는 2D 이미지를 다루기 위해, 원래 이미지를 잘라서 2D 패치를 펼친(flatten) 뒤 시퀀스로 변환하였음.

$$\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$$

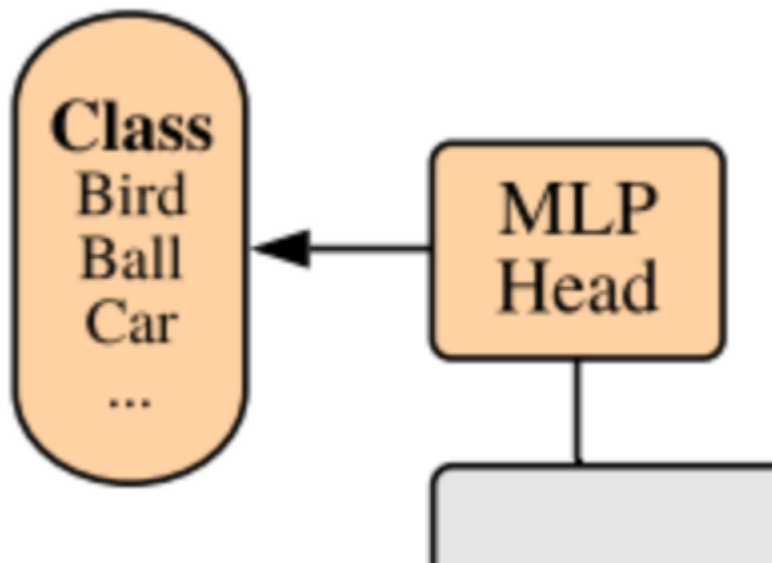
- (H, W) : 원본 이미지의 해상도
- C : 채널 개수
- (P, P) : 각 **image patch**의 해상도
- N : reshape 결과 나오게 되는 **image patches**의 개수

$P^2 * c$ 차원의 latent vector를 갖는, 길이 N 의 효율적인 **input sequence(tokens)**를 형성

하지만, standard transformation에서는 input sequence(그리고 트랜스포머 내에 계속
해 전달되는 query)들은 **D 차원의 latent vector**를 가짐.

그래서, 위에서 말한 $P^2 * c$ 차원의 이미지 패치를 D 차원으로 매핑시키는 **linear
projection** 과정을 추가해줌.

→ 이 때 나온 output이 **patch embeddings**



다음으로 BERT의 [CLS] 토큰처럼, 임베딩된 패치 시퀀스 맨 앞에 학습 가능한 **클래스 토큰**
을 붙인다.

이 [class] 토큰이 최종적으로 이미지 전체를 대표하는 벡터 역할을 함.

→ 사전학습(pre-training)과 파인튜닝(fine-tuning)에서 모두, [class] 토큰의 최종 출력
에 분류기(classification head)를 붙여 사용.

- pre-training : MLP with one hidden layer
[class] 토큰 → (Linear → 활성화함수 → Linear) → softmax
- fine-tuning : MLP with no hidden layer(only single linear layer)
[class] 토큰 → (Linear) → softmax

Patch + Position Embedding →

* Extra learnable
[class] embedding

패치 순서만으로는 원래 위치를 알 수 없으므로, 위치 임베딩(Position Embedding)을 더해 위치 정보를 보존.

→ NLP의 문장 위치 임베딩과 동일 원리.

쉽게 설명하자면,

Transformer는 입력을 시퀀스(토큰 벡터)로 받음.

그런데 Self-Attention은 **순서 정보**를 직접적으로 알지 못함.

- 예: 문장 "나는 학교에 간다" → "학교에 나는 간다"
- 단어들이 같아도, 순서가 바뀌면 의미가 달라지는데, 그냥 토큰 벡터만 있으면 순서를 모름.

→ 그래서 "이 토큰이 몇 번째에 있는지" 알려줄 추가 정보가 필요 → **Positional Embedding**

단어/패치 임베딩에 "나는 1번째, 학교는 2번째"라는 좌표 개념을 부여.

논문에서는 "2D-aware positional embedding을 써봤지만 성능 향상은 크지 않았다"고 보고함.

따라서 ViT는 **learnable 1D positional embedding** 사용.

3.2. Fine-Tuning And Higher Resolution

저자들 또한 NLP에서 했던 것처럼 ViT를 **large dataset**에 **pre-trained**한 다음, **down stream tasks**에 **fine-tuning**을 진행하였음.

▼ 다운스트림이란?

- **사전학습(Pre-training)**: 아주 큰 데이터셋에서 **범용적인 패턴**을 배우는 단계.
 - 예: ViT를 ImageNet-21k (14M 이미지)나 JFT-300M (3억 이미지) 같은 대규모 데이터셋으로 학습.

- **다운스트림 과제(Downstream task):** 사전학습된 모델을 실제로 적용할 구체적이고 작은 과제.

- 예시:

- “고양이 vs 개” 이진 분류 (작은 데이터셋)
- CIFAR-100 (100개 클래스 분류)
- 의료 영상 데이터 분류

이런 down stream task에 적용하기 위해서 pre-train 할 때에 **prediction head**를 없애고, $D \times K$ 의 feed forward layer로 변경을 한다.

이 때 K 는 downstream task의 class 개수이다.

사전 훈련된 ViT 모델은 특정 해상도(예: 224x224)에서 이미지를 고정된 크기의 패치로 잘라 학습한다.

이때 각 패치의 위치 정보를 나타내는 **위치 임베딩**도 이 해상도에 맞춰 훈련된다. 하지만 미세 조정을 위해 더 높은 해상도의 이미지(예: 448x448)를 사용하면, 패치의 개수가 늘어나기 때문에 사전 훈련된 위치 임베딩이 더 이상 맞지 않게 된다.

따라서 ViT가 새로운 고해상도 이미지에서도 패치의 공간적 관계를 이해하도록, 사전 훈련된 위치 임베딩을 2D 보간법(2D Interpolation)을 사용해 새로운 해상도에 맞게 크기를 조절해준다.

→ 이러한 해상도 조정과 보간 작업은 ViT 모델에 2D 이미지 구조에 대한 **귀납적 편향**을 수동으로 주입하는 유일한 부분이다.